Database After Graduation

Week 4

By Narges Yarahmadi

Feb 2026

Table of Contents

Why Concurrency Matters in Real Systems

In academic environments, database queries often execute sequentially.

In production systems, this assumption immediately breaks.

Examples in Banking System: Two withdrawal requests arrive simultaneously:

```
Account Balance = $100
Request A withdraws $80
Request B withdraws $50
```

Without concurrency control, Both requests read:

```
balance = 100
```

Result:

```
100 - 80 = 20
100 - 50 = 50
```

Final stored balance becomes incorrect. This is called **data inconsistency caused by concurrent access**.

Where Concurrency Happens

Modern systems experience concurrency at multiple layers:

- Multiple users
- API requests
- Microservices
- Background workers
- Scheduled jobs
- Distributed servers

Example:

```
Mobile App
Web Client
Admin Dashboard
Background Billing Job
```

All interacting with the same database simultaneously.

Transactions: The Atomic Unit of Work

A transaction is a sequence of one or more operations (reads/writes) executed as a single logical unit of work. Transactions bring **predictability** to state changes .

A **transaction** is a group of database operations executed as a single logical unit.

Either: ✅ Everything succeeds or ❌ Nothing happens

.1 The ACID properties define reliable transaction processing:

Atomicity: All or nothing. If the system crashes mid-process, the transaction is rolled back.

All operations complete or none do. Example in Money transfer:

```
Debit Account A
Credit Account B
```

If credit fails Debit must rollback.

Consistency: A transaction brings the database from one valid state to another, preserving defined rules (constraints, triggers).

Database moves from one valid state to another. Constraints remain preserved. Example:

```
Balance cannot be negative
Foreign keys remain valid
```

Isolation: Concurrent execution of transactions results in a state that is equivalent to some serial (sequential) execution.

Transactions should not interfere with each other. Users should behave as if operations run independently

Durability: Once a transaction is committed, it remains so, even in the event of a power loss or crash. Once committed, Data survives crashes. Handled using:

- Write-ahead logging

- Disk persistence
- Replication

.2 Commit and Rollback: The "Save" and "Undo" Buttons

Managing state requires control over when changes become permanent.

- **BEGIN TRANSACTION:** Marks the starting point.
- **COMMIT:** Saves all modifications made since the `BEGIN`. This makes the changes durable and visible to other transactions (depending on isolation level) .

Makes transaction permanent. Changes become visible.

Example:

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1;

COMMIT;
```

- **ROLLBACK:** Aborts the transaction, undoing all modifications made since the `BEGIN`. This is crucial for error handling .

Undo all operations. Example:

```
ROLLBACK;
```

Used when:

- Errors occur
- Validation fails
- System crashes
- Business rules violated

Industrial Code Example (Error Handling):
In a banking transfer, we debit Account A and credit Account B. If the credit fails, we *must* roll back the debit.

```
-- Pseudo-code for a fund transfer
BEGIN TRY
    BEGIN TRANSACTION TransferFunds;

    UPDATE Accounts SET balance = balance - 100 WHERE id = 'A';
    -- Check for errors, maybe insufficient funds
    UPDATE Accounts SET balance = balance + 100 WHERE id = 'B';

    COMMIT TRANSACTION; -- If both succeed, save permanently
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION; -- If ANYTHING fails, undo the debit
    SELECT 'TRANSACTION FAILED, funds returned';
END CATCH;
```

Takeaway: Never assume a multi-step operation succeeds until the `COMMIT` is acknowledged .

.3 Race Conditions: The Concurrency Nightmare

When multiple threads or processes access shared data concurrently, and the final outcome depends on the timing of their execution, you have a **race condition** .

One of the most common production failures. A race condition occurs when: System outcome depends on execution timing.

Example A: Ticket Booking

Two users purchase last seat. Process:

```
Check available seats
Reserve seat
Confirm purchase
```

Both requests read availability simultaneously. Result is Overselling.

Example B: The "Lost Update" Problem (Bank Account Analogy):
Imagine a bank account with $1000. Two concurrent requests arrive:

1. **Request A (Credit):** Add $100.
2. **Request B (Debit):** Subtract $50.

The Race:

- Thread A reads balance: **1000**
- Thread B reads balance: **1000** (Thread A hasn't written yet)
- Thread A calculates new balance: 1000 + 100 = **1100**
- Thread B calculates new balance: 1000 - 50 = **950**
- Thread A writes: **1100** (Thread B's write overwrites this if done after)
- Thread B writes: **950**

Result: Despite two valid operations, the final balance is $950, not the correct $1050. One update was "lost" .

Real-World Bug (False Mutex):
A common flaw is implementing a lock that only protects part of the system. Consider a batch process (`executeBatch`) that uses a flag `isProcessingBatch` to prevent concurrent runs. However, if the standard API endpoint (`/assignParcel`) ignores this flag, a race condition occurs:

- Batch reads capacity (0/100).
- API reads capacity (0/100) before Batch commits.
- Both assign parcels (50kg + 60kg).
- **Result:** Truck capacity is violated (110kg/100kg) .

.4 Transaction Isolation Levels

To prevent race conditions without forcing all transactions to run one at a time (which kills performance), databases offer Isolation Levels. These levels define how transaction changes are visible to each other. They balance **consistency** against **concurrency** .

Imagine a database handling thousands of transactions at the same time.

If the database forced this:

```
Transaction A finishes completely
then Transaction B starts
then Transaction C starts
```

✅ Perfect correctness ❌ Terrible performance

Modern systems instead allow **transactions to run concurrently**, but they must control **how much transactions can see each other's changes**.

Isolation levels are therefore. **Rules that define what one transaction is allowed to observe from another transaction.**

They balance:

```
Consistency ↔ Performance
Safety      ↔ Speed
```

First, we define the **phenomena** we are trying to prevent:

.4.1 Dirty Read

Reading data written by a concurrent uncommitted transaction . What happens: Transaction reads data that another transaction has not committed yet.

Example: (Not committed yet)

Transaction A:

```
BEGIN;
UPDATE accounts
SET balance = 0
WHERE id = 10;
```

Transaction B

```
SELECT balance FROM accounts WHERE id = 10;
```

B sees:

```
balance = 0
```

Now imagine, Transaction A crashes.

```
ROLLBACK;
```

Actual balance returns to:

```
balance = 500
```

But Transaction B already used wrong data.

Why This Is Dangerous?

This means applications make decisions based on data that never officially existed. Financial systems absolutely forbid this.

.4.2 Non-Repeatable Read:

Reading the same row twice in the same transaction and getting different values, because another transaction modified and committed it in between.

Example A:

Transaction A starts:

```
BEGIN;
```

Reads balance:

```
SELECT balance FROM accounts WHERE id = 1;
```

Result:

```
100
```

Meanwhile Transaction B:

```
SELECT balance FROM accounts WHERE id = 1;
UPDATE accounts
SET balance = 200
WHERE id = 1;

COMMIT;
```

Transaction A reads again:

```
SELECT balance FROM accounts WHERE id = 1;
```

Now result:

```
200
```

Inside the SAME transaction.

Example B:

```
calculate interest
verify funds
approve payment
```

But values changed mid-process.

✅ Non-repeatable read = Same row gives different values.

.4.3 Phantom Read:

Running the same query twice and getting a different set of rows (because another transaction inserted or deleted rows that match the WHERE clause) . It involves **rows appearing or disappearing**.

Transaction A:

```
BEGIN;
```

Query:

```
SELECT * FROM orders
WHERE amount > 1000;
```

Result:

```
5 rows
```

Transaction B inserts:

```
INSERT INTO orders VALUES (... 1500 ...);
COMMIT
```

Transaction A runs same query again:

```
SELECT * FROM orders
WHERE amount > 1000;
```

Now:

```
6 rows
```

A new row appeared like a ghost. Hence, Phantom Read.

This matters in:

- reporting
- inventory checks
- fraud detection
- analytics inside transactions

The Isolation Levels (Core Idea)

Level	Dirty Read	Non-Repeatable	Phantom Read	Use Case & Analogy	Prevents	Performance
Read Uncommitted	Allowed	Allowed	Allowed	**"The Gossip":** Reading data instantly, even if it's not finalized. **High risk**, rarely used in critical financial systems.	❌ Dirty Reads ❌ Non-repeatable reads ❌ Phantom reads	Very High
Read Committed	Prevented	Allowed	Allowed	**Default in PostgreSQL/Oracle.** "The Official Statement." You only see data that has been officially committed. Prevents gossip, but data can change mid-transaction.	✅ Dirty Reads ❌ Non-repeatable reads ❌ Phantom reads	High
Repeatable Read	Prevented	Prevented	Allowed	**Default in MySQL/InnoDB.** "The Snapshot." If you read a row, you see the same value even if someone else changes it. However, new	✅ Dirty Read ✅ Non-repeatable Read ❌ Phantom Reads (conceptually)	Medium

				"phantom" rows can appear.		
Serializable	Prevented	Prevented	Prevented	**"The Vault."** Transactions execute as if they were queued one after another. Highest consistency, lowest concurrency.	✅ Dirty Read ✅ Non-repeatable Read ✅ Phantom Reads (conceptually)	Lower

How to Set It:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Tools of the Trade: Managing Concurrency & Consistency

Databases provide transactions and isolation levels. But in modern systems, most concurrency mistakes actually happen in the **application layer**, not inside SQL.

Because today's systems look like this:

```
API Server
Business Logic
Repositories
ORM
Database
```

So consistency must be controlled **above the databa**se as well. Transaction logic doesn't just live in SQL scripts; it lives in your application code.

.1 Application-Level Transaction Management

Imagine a real backend operation. For example, creating a task system like Jira or Trello. When creating a task you might need to:

1. Insert Task
2. Insert Comments
3. Update Project statistics
4. Log activity

If step 3 fails but steps 1 and 2 already succeeded, your database becomes inconsistent. Example disaster:

Task exists
Comments exist
Project counter incorrect

This is why we need, one transaction covering the entire business operation.

.2 Why Not Just Write SQL Transactions Everywhere?

You *could* do this:

```
BEGIN;
INSERT ...
UPDATE ...
INSERT ...
COMMIT;
```

But modern applications:

- use ORMs
- multiple repositories
- service layers
- async APIs

Developers easily forget:

```
commit()
rollback()
error handling
```

Human error becomes the biggest risk.

.3 The Repository Pattern & Transactor Pattern (First Idea):

Repositories are responsible for separating and organizing database access within an application. For example, components such as **TaskRepository**, **CommentRepository**, and **UserRepository** each handle interactions related to their specific data domain. Every repository understands how to query the database, the structure of the underlying tables, and the

ORM logic required to map application objects to database records. However, repositories operate independently and do not manage or coordinate transactions across multiple operations.

To address this limitation, the **Transactor Pattern** introduces a clear transaction boundary that ensures consistency during complex operations. It can be viewed as a supervisory layer overseeing the entire unit of work. The transactor controls the transaction lifecycle by starting a transaction, executing the required business logic, and then deciding whether to commit or roll back the changes. If all operations succeed, the transaction is committed. If any step fails, the system automatically performs a rollback.

In practice, when code executes a block such as **repositoryTransactor.run(async () => { ... }),** a database transaction is implicitly started using a **BEGIN TRANSACTION** command. Repository operations inside this block, such as inserting a task, are executed within that same transaction context. If an error later occurs, such as a validation failure, network issue, or another repository operation failing, the transactor automatically triggers a **ROLLBACK**, ensuring that all partial changes are safely discarded and the database remains consistent.

To ensure consistency across multiple repository updates (e.g., updating a `Task` and its `Comments`), we define a boundary .

Concept: The **Transactor** wraps the entire unit of work. If any repository operation fails, the transactor triggers a rollback for all of them .

Industrial Example (Pseudo-code for a Service Layer):

typescript

```
// A Use Case for creating a Task
class CreateTaskUseCase {
  // Inject the Transactor, not the raw connection
  constructor(
    private taskRepository: TaskRepository,
    private repositoryTransactor: RepositoryTransactor
  ) {}

  async handle(request: CreateTaskRequest) {
    // The Transactor ensures atomicity
    return this.repositoryTransactor.run(async () => {
      // Business logic
      const task = Task.create(request.taskName);
```

```
    // Persist the aggregate
    await this.taskRepository.insert(task);

    // If this fails, the Transactor automatically rolls back the insert
    return task;
  });
 }
}
```

In this pattern, the developer doesn't need to remember to call commit() or rollback(). The Transactor handles it automatically, reducing human error .

Industry strongly favors the **Transactor Pattern** because it removes the burden of manual transaction management from developers. Instead of explicitly handling commits and rollbacks in multiple places, the framework manages the process automatically. This significantly reduces common and costly issues such as forgotten rollbacks, partial database writes, or inconsistent and dangling data. As a result, this pattern has become widely adopted in modern backend ecosystems including NestJS, Spring Boot, Go-based services, and many systems designed using Domain Driven Design principles.

From a practical mental model perspective, systems without a transactor place full responsibility for transaction safety on the developer, which increases the likelihood of errors in complex workflows. When a transactor is used, transaction control becomes a framework responsibility rather than an individual coding concern. This shift lowers operational risk and allows developers to focus primarily on business logic while ensuring that database consistency is maintained automatically.

An important principle in industrial system design is that the **transaction boundary should exist at the Use Case level rather than inside repositories**. A use case such as *CreateOrderUseCase* represents a complete business operation that must succeed or fail as a whole. Placing transactions inside smaller repository methods, such as separate insertOrderTransaction() or updateInventoryTransaction() functions, fragments the process into multiple independent transactions. This breaks atomicity and can lead to inconsistent system states when one operation succeeds while another fails.

.4 API-Based Concurrency Testing

A fundamental question in backend engineering is **how to prove that a system is free from race conditions**. The answer is to simulate real world chaos using **API based load and concurrency testing**. Race conditions rarely appear during development because testing usually happens with a single request or a small number of users. In real production environments,

however, hundreds of users may interact with the system at the same moment. Instead of manually sending one request at a time, engineers intentionally stress the application by sending many concurrent requests to expose hidden synchronization and consistency problems.

To achieve this, industry teams rely on specialized testing tools designed to simulate realistic traffic patterns. Commonly used tools include:
• **Apache JMeter**: The industry standard, heavily used in banking and telecom systems, capable of simulating thousands of users, workflows, and authenticated sessions through a graphical interface.
• **K6**: A modern, scriptable load testing tool written using JavaScript and widely integrated into CI/CD pipelines.
• **CLI or Go based Benchmark Tools**: Lightweight command line tools used to quickly hammer API endpoints without complex setup.

The testing strategy is simple but powerful. Instead of sending a single request, engineers send many requests simultaneously to reproduce real user behavior. For example:
• `-c 100` means **100 concurrent users sending requests at the same time**.
• `-n 1000` means **1000 total requests during the experiment**.
• Testing endpoints such as `/api/transfer` is especially effective because transactional operations are highly sensitive to race conditions.

Interpreting the results correctly is critical. Consider a test result showing 998 successful requests and 2 failures. A beginner might assume this indicates good performance, but engineers immediately investigate any failure. If the response is **HTTP 409 Conflict**, it often means the database successfully prevented an unsafe concurrent update, such as two users modifying the same record simultaneously through optimistic locking. This may represent a positive outcome where transaction safety protected the system. However, it can also signal deeper issues such as deadlocks or logical race conditions, which require careful log analysis. In industrial systems, the objective is always safe failure, meaning a controlled error is far preferable to allowing corrupted operations such as an incorrect financial transfer.

Example using a CLI tool:

bash

```
# Simulate 100 concurrent users ( -c 100 ) making a total of 1000 requests ( -n 1000 )
# to the /api/transfer endpoint
./benchmark -c 100 -n 1000 -u http://localhost:3000/api/transfer -m POST

# Output Analysis:
# Successful requests: 998
# Failed requests: 2 <-- This is suspicious. Did we get a deadlock? A constraint violation?
#
```

```
# Status code distribution:
#   200: 998
#   409: 2   <-- HTTP 409 Conflict suggests the database prevented a duplicate/lost update
```

Interpretation: If your test shows failures under high concurrency (e.g., HTTP 409s or 500s), you have likely exposed a race condition that your optimistic locking or transaction isolation successfully caught (good) or failed to handle (bad) .

For better understanding, Try the activity.pdf file and test and see the results.

.5 Read Replicas and Consistency Trade-offs

As systems scale, the primary database gets overloaded with read queries. A common solution is **Read Replicas**, copies of the database that handle only `SELECT` queries .

The Architecture:

- **Primary (Writer):** Handles all `INSERT`, `UPDATE`, `DELETE`.
- **Replicas (Readers):** Asynchronously copy changes from the primary.

The Consistency Trade-off (The Replication Lag Problem):
Replication is usually **asynchronous** (for performance). This means a replica is always slightly behind the primary .

The Bug Scenario (Causal Dependency):

1. **User A (Write):** Posts a comment on a blog post. (Goes to Primary).
2. **User A (Read):** Is redirected to the comment page.
3. **Load Balancer:** Sends the read request to a **Read Replica** that hasn't received the new comment yet.
4. **User A sees:** "Your comment was posted!" but the page below shows no comments.
5. **Result:** The user sees inconsistent data, breaking the user experience .

Solving the Problem:

- **Session/Read-after-Write Consistency:** The application must remember that User A just wrote something. For the next 5-10 seconds, route *their* reads to the Primary, not the Replica .

- **GTID Consistent Reads:** Advanced setups (like MySQL with GTID) allow the replica to wait until it has caught up to the specific transaction ID before returning data to the user .

Summary

- **Transactions** (COMMIT/ROLLBACK) provide atomicity to prevent partial failures.
- **Race Conditions** occur when concurrent operations interleave incorrectly, leading to lost updates.
- **Isolation Levels** (Read Committed, Serializable) are tools to balance performance against consistency guarantees.
- **Read Replicas** scale read throughput but introduce **eventual consistency**, requiring application-level logic to handle stale reads.